

Formati di stampa e do/while

Scrivere i numeri come vogliamo; rileggere un dato finché è valido

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Come stampo un numero come voglio io?

formati.c, passo 1: %f, larghezza, decimali, tabulazione

formati.c (passo 1)

```c
int main() {
    double x = 12345678.123456789;
    double y = 0.0001234567;
    printf("formato x\t\ty\n");
    printf("%%f      %f\t%f\n", x, y);
    printf("%%5.3f   %5.3f\t%5.3f\n", x, y);
}
```

Terminale

```
$ ./app.exe
formato x                y
%f      12345678.123457 0.000123
%5.3f   12345678.123    0.000
```

%f

6 decimali: 0.0001234567 diventa 0.000123

%5.3f

almeno 5 caratteri e 3 decimali: 0.000

\t

tabulazione: la colonna successiva

%%

stampa un solo %: il % introduce i formati

E se il numero è enorme o minuscolo?

formati.c, passo 2: notazione scientifica, %g e le cifre in più

formati.c (finale)

```c
int main() {
    double x = 12345678.123456789;
    double y = 0.0001234567;
    printf("formato x\t\ty\n");
    printf("%%f      %f\t%f\n", x, y);
    printf("%%5.3f   %5.3f\t%5.3f\n", x, y);
    printf("%%5.3e   %5.3e\t%5.3e\n", x, y);
    printf("%%5.3g   %5.3g\t%5.3g\n", x, y);
    printf("%%.15f   %.15f\t%.15f\n", x, y);
}
```

Terminale

```
$ ./app.exe
formato x                y
%f      12345678.123457 0.000123
%5.3f   12345678.123    0.000
%5.3e   1.235e+07       1.235e-04
%5.3g   1.23e+07        0.000123
%.15f   12345678.123456789180636        0.000123456700000
```

%e

notazione scientifica: 1.235e+07 vale $1.235 \cdot 10^7$

%g

sceglie da solo la forma più corta

%.15f

le ultime cifre sono errore di rappresentazione

E se l'utente scrive un voto sbagliato?

voto.c, oggi: il programma controlla e finisce. Vogliamo che lo richieda

voto.c (con solo if)

```c
#include <stdio.h>

int main() {
    int voto;
    printf("voto (0-30): ");
    scanf("%d", &voto);
    if (voto < 0 || voto > 30) {
        printf("voto non valido\n");
    } else {
        printf("voto %d registrato\n", voto);
    }
}
```

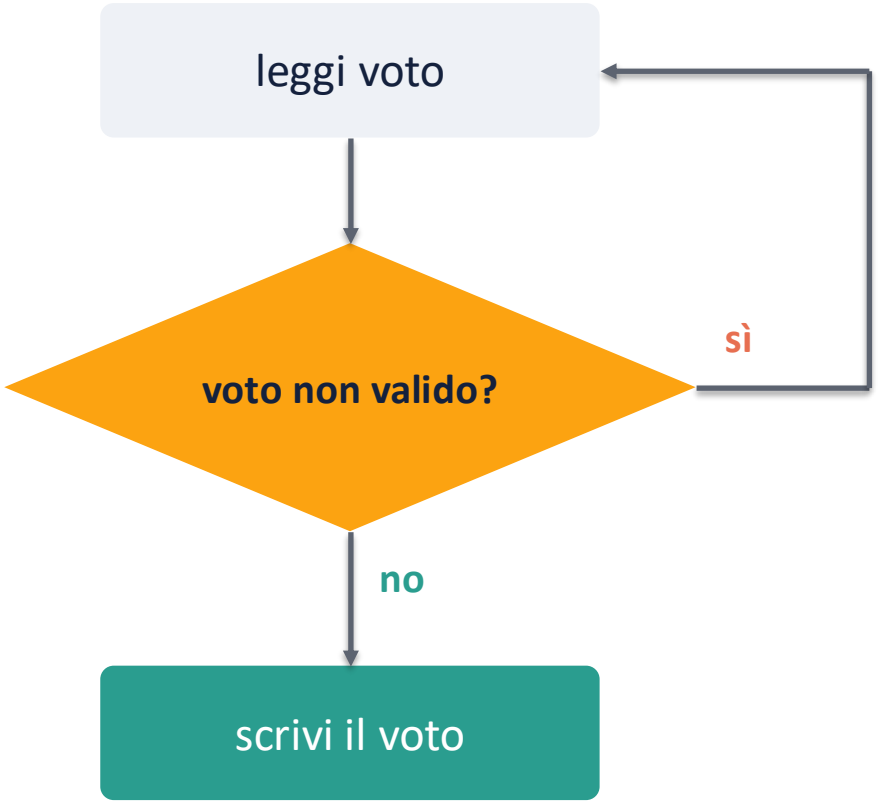
Terminale

```
$ ./app.exe
voto (0-30): 27
voto 27 registrato
$ ./app.exe
voto (0-30): 45
voto non valido
```

Il controllo c'è, ma il programma finisce: l'utente deve rilanciarlo. Vogliamo **ripetere la domanda finché il voto è valido**.

Ripeti finché il dato non è valido

Il diagramma di flusso: si legge prima, si controlla dopo



in pseudocodice

```
do {
    leggi voto
} while (voto non valido);
```

Almeno una volta

il voto si legge sempre; la condizione si controlla dopo

Finché la condizione è vera

se voto non valido è vero, il ciclo ricomincia

Prima versione: la variabile dentro le graffe

voto.c non compila: una variabile dichiarata dentro { } esiste solo lì dentro

voto.c (prima versione)

```
int main() {
    do {
        int voto;
        printf("voto (0-30): ");
        scanf("%d", &voto);
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
}
```

Terminale

```
voto.c:8:14: error: use of undeclared
identifier 'voto'; did you mean 'goto'?
    } while (voto < 0 || voto > 30);
             ^~~~
```

voto è dichiarata dentro il do { }: quando si arriva alla condizione del while, fuori dalle graffe, non esiste più. Si dichiara prima del do.

do { … } while (condizione);

voto.c: la variabile fuori dal do, la condizione dopo le graffe

voto.c

```c
int main() {
    int voto;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
}
```

Terminale

```
$ ./app.exe
voto (0-30): 45
voto (0-30): -3
voto (0-30): 27
voto 27 registrato
```

int voto;

fuori dal do: esiste anche nella condizione

do { … }

il corpo si esegue almeno una volta

while (…);

ripete finché è vera: il ; finale è obbligatorio

Cosa succede a ogni giro

Traccia a mano di voto.c con i valori 45, -3, 27

giro	voto letto	condizione del while	ripete?
1	`45`	`45 < 0 || 45 > 30  =>  0 || 1  =>  1`	sì: legge di nuovo
2	`-3`	`-3 < 0 || -3 > 30  =>  1 || 0  =>  1`	sì: legge di nuovo
3	`27`	`27 < 0 || 27 > 30  =>  0 || 0  =>  0`	no: esce dal ciclo

Dopo l'ultimo giro: la condizione vale 0, il ciclo finisce e il programma stampa «voto 27 registrato».

Quanti tentativi sono serviti?

tentativi.c: una seconda variabile, il contatore

tentativi.c

```c
int main() {
    int voto = -1;          // -1 = non valido
    int ntentativi = 0;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
        ntentativi = ntentativi + 1;
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
    printf("tentativi: %d\n", ntentativi);
}
```

Terminale

```
$ ./app.exe
voto (0-30): 45
voto (0-30): -3
voto (0-30): 27
voto 27 registrato
tentativi: 3
$ ./app.exe
voto (0-30): 28
voto 28 registrato
tentativi: 1
```

ntentativi = ntentativi + 1;

somma 1 al valore che c'è già: a ogni giro

int ntentativi = 0;

parte da 0, prima del do: si usa dopo il ciclo

E se scrive una lettera invece di un numero?

Lo stesso programma, con l'input abc: il ciclo non finisce più

tentativi.c

```c
int main() {
    int voto = -1;          // -1 = non valido
    int ntentativi = 0;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
        ntentativi = ntentativi + 1;
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
    printf("tentativi: %d\n", ntentativi);
}
```

Terminale

```
$ ./app.exe
voto (0-30): abc
voto (0-30): voto (0-30):
voto (0-30): voto (0-30):
voto (0-30): voto (0-30):
^C
```

scanf non trova un numero: non scrive in voto (resta -1) e non consuma la lettera. La condizione resta vera e il ciclo gira per sempre. Si interrompe con **`Ctrl-C`**.

Due errori da riconoscere

Manca il ; finale, oppure la condizione è al contrario

senza ; dopo il while (non compila)

```
int main() {
    int voto;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
    } while (voto < 0 || voto > 30)
    printf("voto %d registrato\n", voto);
}
```

Terminale

```
voto.c:8:36: error: expected ';' after do/while statement
    } while (voto < 0 || voto > 30)
                                   ^
```

condizione invertita (compila)

```
int main() {
    int voto;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
    } while (voto >= 0 && voto <= 30);
    printf("voto %d registrato\n", voto);
}
```

Terminale

```
$ ./app.exe
voto (0-30): 27
voto (0-30): 12
voto (0-30): 45
voto 45 registrato
```

Esercizi

Da fare a casa, poi si controlla con un programma in C

1 Temperatura

Chiedi la temperatura (in °C) di un paziente e ripeti la domanda finché non è tra 30 e 45. Alla fine stampa il valore con un decimale e il numero di tentativi.

prova con:

```
36.6
50    20    38.5
```

2 Dispari positivo

Chiedi un intero n e ripeti finché non è positivo e dispari.

prova con:

```
8    -3    7
4    9
```

3 Energia cinetica

Leggi m (kg) e v (m/s) e calcola E = ½ m v². Stampa E in due modi: con %.3e e con %g. Attenzione a come scrivi ½.

prova con:

```
m v
0.145 40
1200 25
9.1e-31 2e6
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (-lm serve per le funzioni di math.h)

Riepilogo

1	**Formati**	%f %5.3f %e %g %.15f, \t e %%: scegliere come scrivere un numero
2	**do / while**	il corpo si esegue almeno una volta; si ripete finché la condizione è vera; variabile dichiarata fuori
3	**Dato non valido**	condizione = caso da correggere; lettera al posto del numero: ciclo infinito, Ctrl-C



while e la caduta del grave

Controllare prima di ripetere; costanti con #define; una caduta libera

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Conto alla rovescia: e se n è già zero?

conto.c con do/while: il corpo si esegue sempre almeno una volta

conto.c (con do/while)

```c
#include <stdio.h>

int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);
    do {
        printf("%d\n", n);
        n = n - 1;
    } while (n > 0);
    printf("via!\n");
}
```

Terminale

```
$ ./app.exe
n = 3
3
2
1
via!
$ ./app.exe
n = 0
0
via!
```

Con n = 0 stampa 0 e poi «via!»: il corpo è già stato eseguito. Serve un ciclo che controlli la condizione prima.

Controllare prima: while

conto.c: la condizione sta prima del corpo, che può non essere eseguito mai

conto.c (con while)

```c
#include <stdio.h>

int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);
    while (n > 0) {
        printf("%d\n", n);
        n = n - 1;
    }
    printf("via!\n");
}
```

Terminale

```
$ ./app.exe
n = 3
3
2
1
via!
$ ./app.exe
n = 0
via!
```

while (cond) { … }

controlla prima: il corpo può non eseguirsi mai

do { … } while (cond);

controlla dopo: il corpo si esegue almeno una volta

while (n > 0);

il ; chiude il ciclo con il corpo vuoto: ciclo infinito

Scrivere n = n - 1 diventa noioso

*abbreviazioni.c: ++, --, +=, -=, *=*

abbreviazioni.c

```c
int main() {
    int i = 5;
    i++;                        // come i = i + 1
    printf("i++   %d\n", i);
    i += 4;                     // come i = i + 4
    printf("+= 4  %d\n", i);
    i *= 3;                     // come i = i * 3
    printf("*= 3  %d\n", i);
    i--;                        // come i = i - 1
    printf("i--   %d\n", i);
    i -= 9;                     // come i = i - 9
    printf("-= 9  %d\n", i);
}
```

Terminale

```
$ ./app.exe
i++   6
+= 4  10
*= 3  30
i--   29
-= 9  20
```

i++ i--

come i = i + 1 e i = i - 1

i += 4 i -= 9

come i = i + 4 e i = i - 9

i *= 3

come i = i * 3

Dove si trova il sasso dopo t secondi?

Lasciato cadere da fermo da un'altezza h: quota y(t) in funzione del tempo

$$y(t) = h - \tfrac{1}{2} \cdot g \cdot t^2$$

t (s)	y(t), con h = 20 m	y (m)
0.0	20 − 0.5 · 9.81 · 0.0²	20.00
0.5	20 − 0.5 · 9.81 · 0.5²	18.77
1.0	20 − 0.5 · 9.81 · 1.0²	15.09

l'idea del programma

```
t = 0;   y = h;
finché y > 0:    calcola y;   stampa t e y;   t += dt;
```

Un valore che compare ovunque: #define

costante.c: GRAV vale 9.81 in tutto il programma e non può cambiare

costante.c

```c
#include <stdio.h>
#define GRAV 9.81

int main() {
    double m, peso;
    printf("massa (kg): ");
    scanf("%lf", &m);
    peso = m * GRAV;
    printf("peso: %6.2f N\n", peso);
}
```

Terminale

```
$ ./app.exe
massa (kg): 70
peso: 686.70 N
```

costante.c (con un'assegnazione, non compila)

```c
    GRAV = 1.62;
```

Terminale

```
error: expression is not assignable
```

Il programma della caduta

grave.c: una tabella tempo-quota, con while e #define

grave.c

```c
#include <stdio.h>
#define GRAV 9.81

int main() {
    double h, dt, t, y;
    printf("altezza h (m): ");
    scanf("%lf", &h);
    printf("passo dt (s): ");
    scanf("%lf", &dt);
    t = 0.;
    y = h;
    printf("    t\t    y\n");
    while (y > 0.) {
        y = h - 0.5 * GRAV * t * t;
        printf("%6.2f\t%6.2f\n", t, y);
        t += dt;
    }
}
```

Terminale

```
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.5
    t        y
  0.00    20.00
  0.50    18.77
  1.00    15.09
  1.50     8.96
  2.00     0.38
  2.50   -10.66
```

`%6.2f\t%6.2f`: colonne allineate `t += dt;`: il tempo avanza `while (y > 0.)`: si ferma se y non è positiva

L'ultima riga è sotto il suolo

Il passo dt è un'approssimazione: la tabella salta il momento dell'impatto

Terminale (t e y dell'ultima riga)

```
h = 20 m: ultima riga per tre passi

dt = 0.5      2.50  -10.66
dt = 0.1      2.10   -1.63
dt = 0.01     2.02   -0.01
```

Tempo di caduta: 2.02 s

√(2h/g): il sasso tocca terra tra 2.00 e 2.50

Errore del passo

più dt è piccolo, più l'ultima quota è vicina a 0

Da discutere

come evitare di stampare la riga sotto il suolo, senza cambiare il resto del programma

Perché succede

il while controlla y dopo averla già stampata: l'ultimo passo oltrepassa y = 0

Esercizi

Da fare a casa, poi si controlla con un programma in C

1 Sulla Luna

Il sasso cade sulla Luna (g = 1.62 m/s², con #define). Leggi h e dt e stampa la tabella t e y, senza la riga sotto il suolo.

prova con:

```
h dt
15 0.5
8 0.25
```

2 Somma che cresce

Leggi un limite e somma 1 + 2 + 3 + … finché la somma non supera il limite. Stampa quanti termini sono serviti e la somma.

prova con:

```
100
1000
5
```

3 Potenze di 2

Leggi un limite e stampa, una per riga, le potenze di 2 minori o uguali al limite, usando *=.

prova con:

```
100
1000
1
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (-lm serve solo con math.h)

Riepilogo

1	**while**	controlla la condizione prima: il corpo può non eseguirsi mai
2	**++ -- += -= *=**	n-- vale n = n - 1; t += dt vale t = t + dt; p *= 2 vale p = p * 2
3	**#define**	costante con nome in maiuscolo, senza = e senza ;: il preprocessore la sostituisce
4	**Caduta del grave**	tabella tempo-quota con %6.2f e \t; con dt grande l'ultima riga va sotto il suolo